TALLER DE INTEGRACIÓN II

Preparar microservicio y modelos de publicaciones

Revisión técnica de la base independiente

Proyecto Roomies - ¿Hay cupo? | Versión declarada: 1.0.0

Código revisado: commit 63b8493. El repositorio estaba sin cambios pendientes al comenzar esta revisión. Se revisaron los archivos del servicio, sus consumidores y sus pruebas; no se modificó código de la aplicación.

VEREDICTO	ALCANCE
Correcto como preparación de semana 1. No se detectó un bloqueo actual para esa entrega.	Aplicación FastAPI independiente, endpoint de estado, modelos SQLAlchemy y contratos de validación. La persistencia propia sigue pendiente.

Lo que está bien

- La aplicación carga sin DATABASE_URL y no importa ni inicia la API principal.
- GET /health responde 200; la documentación y la versión 1.0.0 son coherentes.
- Los modelos Publicacion e ImagenPublicacion resuelven su relación y generan SQL PostgreSQL válido.

La aclaración importante

Esto todavía no es un microservicio de publicaciones con operaciones y base de datos propias. Solo expone /health. Crear, consultar, editar, cerrar y eliminar continúan en main/main.py. El README lo declara expresamente, por lo que no se califica como un error de esta preparación.

Sí hay diferencias entre los modelos propuestos y PostgreSQL: tipo de precio, nulabilidad y una clave foránea ausente en la base. Deben resolverse antes de activar la persistencia con el ORM, que es la capa que transforma objetos Python en operaciones SQL.

Qué incluye el informe

Ubicación de archivos; revisión de aplicación, modelos y validación; contraste de estructura real; pendientes concretos; resultados de pruebas y comandos para ejecutar la base del servicio.

02 / REVISIÓN ESPECÍFICA

Carpetas, archivos y responsabilidades

Todas las rutas son relativas a la raíz del repositorio. Las líneas corresponden al commit revisado; pueden cambiar en futuras ediciones.

Archivo y referencia	Qué hace / evaluación
publicaciones_service/ __init__.py Línea 1: Preparación	Identifica el paquete. No inicia conexiones, tablas ni servidores. Correcto.
publicaciones_service/ main.py Línea 4: app = FastAPI	Define la aplicación independiente y /health. Título y versión presentes; no depende de la API principal.
publicaciones_service/ models.py Línea 18: class Publicacion	Define Base, Publicacion e ImagenPublicacion bajo el esquema haycupo. Preparación válida; estructura propuesta.
publicaciones_service/ schemas.py Línea 20: class PreferenciasMascotas	Validación de mascotas y contratos de creación/respuesta. Parte de estos contratos se comparte con la API actual.
publicaciones_service/ requirements.txt Línea 1: fastapi	FastAPI, Uvicorn, SQLAlchemy 2, Psycopg y Pydantic 2. Contiene lo necesario para esta base.
publicaciones_service/ README.md Línea 58: ## Alcance de esta semana	Aclara que conexión y traslado de operaciones corresponden a la siguiente etapa. Comando de inicio correcto desde la raíz.
publicaciones_service/migrations/ 001_mascotas_aceptadas.sql Línea 3: BEGIN	Agrega solo las columnas de mascotas a la tabla existente. No crea el esquema completo ni alinea todas las restricciones del ORM.
tests/ test_publicaciones.py Línea 25: test_modelos_resuelven	Prueba la relación y generación SQL. En el mismo archivo se verifica /health y la validación compartida.

Separación actual

```
publicaciones_service/   aplicación base y modelos propuestos
main/main.py             API que todavía guarda los avisos
PaginaPrincipal/          interfaz que usa la API principal
Creacion_De_publicacion/  formulario de crear y editar
tests/                    pruebas automatizadas
output/pdf/               informes de revisión
```

La aplicación independiente se ejecuta en 8001 según la guía. La web y la API actual usan 8000. Son procesos y alcances distintos; iniciar 8001 no reemplaza automáticamente al backend de la página.

03 / REVISIÓN ESPECÍFICA

Revisión de los modelos SQLAlchemy

Campo(s) de Publicacion	Definición propuesta / observación
id_publicacion / id_usuario	Enteros; id_publicacion es clave primaria. id_usuario identifica al autor, sin una relación ORM con usuarios en este paquete.
titulo / ubicacion	String(200) y String(250), no nulos. Límites coherentes con el contrato preparado.
descripcion / contenido	Text; descripcion obligatoria y contenido opcional para compatibilidad con la API actual.
precio	Numeric(14, 2), no nulo: tipo decimal propuesto para el precio. No convierte el tipo real de la tabla existente.
categoria / tipo / tipo_vivienda	String no nulo. Los valores permitidos se validan en los contratos de entrada; no hay CHECK de dominio en este modelo.
estado / cupos_totales	String con default Disponible; entero obligatorio para cupos. El mínimo de 1 se controla en la validación de entrada, no con un CHECK del ORM.
mascota	String obligatorio con default no. Se conserva la respuesta si/no de la aplicación.
tipos_mascotas_aceptadas	ARRAY(String(40)), obligatorio y default=list. La función list evita compartir una misma lista entre inserciones.
condiciones_mascotas	Text opcional para condiciones de aceptación.
permitir_comentarios	Boolean obligatorio con default=False.

ImagenPublicacion y relación

id_imagen es clave primaria; **id_publicacion** es una clave foránea obligatoria hacia haycupo.publicacion.id_publicacion. **url** es obligatoria y **orden** es entero con default 0. La relación usa back_populates en ambas direcciones.

```
Publicacion (1)  <---->  (N) ImagenPublicacion
    .imagenes                .publicacion
```

Se comprobó que configure_mappers() resuelve ambos modelos y que las dos tablas compilan a DDL PostgreSQL. DDL es el SQL que describe tablas y restricciones; generarlo no significa ejecutarlo en una base.

Los default= del modelo son valores aplicados por SQLAlchemy al insertar. No equivalen por sí solos a un DEFAULT en el servidor ni a valores ya asignados al construir el objeto.

04 / REVISIÓN ESPECÍFICA

Validación y contratos: qué se aplica hoy

PublicacionCrear

Prepara un contrato de entrada con normalización de espacios y validación de los datos. No está asociado a un POST del servicio independiente, porque esa ruta todavía no existe.

Dato	Validación preparada
Título / descripción / ubicación	Texto de 1 a 200 / 5000 / 250 caracteres respectivamente; recorta espacios exteriores.
Precio	Decimal no negativo; hasta 14 dígitos y 2 decimales.
Cupos	Entero entre 1 y 2147483647.
Categoría	busco u ofrezco.
Tipo / vivienda	Arrendador o Compartida; casa, departamento o habitacion.
Comentarios	Booleano; por defecto false.

PreferenciasMascotas

• mascota admite si/no; con no se limpian especies y condiciones.

• Si acepta mascotas, exige al menos un tipo. La lista elimina duplicados y admite siete especies/categorías conocidas.

• Elegir otro requiere aclararlo en condiciones, con máximo 2000 caracteres.

• preferencias_desde_formulario interpreta la lista JSON y conserva los avisos antiguos que no especificaban especies, cuando no se modifican esas preferencias.

Qué comparte la API actual

main/main.py utiliza PublicacionCreada y PublicacionResumen como contratos de respuesta y preferencias_desde_formulario para mascotas. La entrada actual se valida mediante parámetros Form de FastAPI; no instancia directamente PublicacionCrear.

PublicacionResumen incluye estado, cupos_totales y cupos_disponibles, además de datos de presentación y mascotas. El precio de salida es float, mientras el contrato de creación usa Decimal. No debe confundirse el formato de respuesta con el tipo de almacenamiento deseado.

Los modelos SQLAlchemy no ejecutan automáticamente los validadores de Pydantic. Cuando se trasladen las operaciones al microservicio, sus rutas deberán validar primero y después guardar los objetos.

05 / REVISIÓN ESPECÍFICA

Comparación con PostgreSQL: diferencias reales

Se revisaron únicamente tipos, restricciones y valores por defecto de las dos tablas mediante una conexión de solo lectura. No se consultaron publicaciones ni se modificaron datos.

Elemento	Modelo preparado	Base observada
Esquema y tablas	haycupo.publicacion e imagen_publicacion	Las dos tablas existen con esos nombres.
Claves primarias	id_publicacion e id_imagen	Presentes; enteros con secuencia.
precio	Numeric(14, 2), NOT NULL	double precision, permite NULL.
titulo / ubicacion	String con límite 200/250, NOT NULL	varchar sin límite declarado, permite NULL.
descripcion	Text, NOT NULL	varchar, permite NULL.
estado / cupos / autor	No nulos	Permiten NULL.
Imagen: publicación / URL / orden	No nulos; orden default 0	Permiten NULL y no tienen DEFAULT declarado.
FK imagen -> publicación	Declarada en el modelo	No se observó esa clave foránea en la tabla real.
Tipos de mascotas	ARRAY de varchar(40), no nulo	ARRAY de varchar, no nulo y con DEFAULT de lista vacía.
Condiciones de mascotas	Text opcional	Text opcional; coincide.

Hallazgo 1: alinear estructura antes de usar el ORM

Qué es: el modelo es una propuesta, no una copia exacta de la estructura actual. **Problema:** declarar Numeric, NOT NULL o ForeignKey en Python no añade esas restricciones a una tabla ya creada.

Corrección mínima: elegir la estructura definitiva, revisar datos antiguos y preparar una migración explícita que alinee tipos y restricciones. Añadir la clave foránea solo después de comprobar que no hay imágenes sin publicación válida.

Si se omite: el modelo y la base tendrán reglas distintas. La base seguirá aceptando datos que el diseño supone imposibles y el precio continuará almacenado en coma flotante.

La migración 001_mascotas_aceptadas.sql solo cubre las dos columnas de mascotas. No soluciona las diferencias anteriores. No se ejecutó ninguna migración durante esta revisión.

06 / REVISIÓN ESPECÍFICA

Pendientes concretos antes de guardar con ORM

Estos casos no rompen el flujo actual: la aplicación principal escribe SQL directo. Son condiciones que deben atenderse al trasladar la persistencia al microservicio.

2. Cambios internos en la lista de mascotas

Ubicación: publicaciones_service/models.py:37. **Qué es:** un ARRAY normal de SQLAlchemy. **Problema:** sobre una lista cargada, append("ave") no marca el atributo como modificado; se reprodujo con el historial de atributos.

Corrección: asignar una lista nueva al actualizarla o configurar MutableList.as_mutable(ARRAY(...)). **Si se omite:** un cambio hecho dentro de la lista podría no guardarse al confirmar la sesión.

3. Borrar una publicación con imágenes

Ubicación: publicaciones_service/models.py:48. **Qué es:** la relación utiliza la cascada predeterminada merge,save-update. **Problema:** no define que borrar un padre también borre sus imágenes; el ORM puede intentar dejar la referencia hija en NULL.

Corrección: decidir eliminación explícita de hijos o una cascada coherente con la futura clave foránea y el cierre sin borrado. **Si se omite:** con la FK no nula propuesta podría fallar el borrado; con la base actual podrían quedar imágenes sin padre.

4. Dos imágenes con el mismo orden

Ubicación: publicaciones_service/models.py:51. **Qué es:** las imágenes se ordenan solo por orden. **Problema:** dos fotos con orden=0 no tienen desempate, mientras la API actual utiliza orden, id_imagen.

Corrección: añadir id_imagen como segundo criterio. **Si se omite:** la futura primera imagen del ORM podría diferir de la portada que hoy muestra la API.

Mejora menor: instalaciones reproducibles

requirements.txt deja FastAPI, Uvicorn y Psycopg sin cotas. El entorno revisado funciona, pero una instalación futura podría resolver versiones diferentes. Registrar versiones verificadas o un archivo de bloqueo mejora la repetición del entorno; no se detectó un fallo actual por este punto.

Prioridad para esta entrega

Preparación: aceptable con el alcance documentado. **Antes de activar persistencia:** resolver 1 y definir 2-4, con pruebas de integración. No se añadieron conexión, repositorios, rutas ni migraciones para anticipar una etapa que aún no se está implementando.

07 / REVISIÓN ESPECÍFICA

Pruebas ejecutadas y alcance de la evidencia

53 pruebas Python del proyecto pasaron. No son 53 pruebas exclusivas del microservicio. La distribución real es:

Grupo	Cantidad	Qué comprueba
Relación ORM y DDL	1	Construye la relación y genera SQL para las dos tablas.
Validación compartida	11	Mascotas, preferencias antiguas y contrato de creación.
API principal simulada	38	Flujos actuales de publicaciones, filtros, edición, cierre e imágenes.
Health independiente	1	Estado y versión del servicio en preparación.
Configuración de main/auth	2	Conexiones de la aplicación principal y autenticación.

Seis comprobaciones específicas adicionales

• Importación independiente con variables de entorno vacías.

• GET /health con 200 y cuerpo exacto: servicio=publicaciones, estado=ok.

• /docs y /openapi.json con 200; título y versión 1.0.0 correctos.

• OpenAPI solo declara /health; /api/publicaciones responde 404 en este servicio.

• Mappers y SQL PostgreSQL de ambas tablas resueltos correctamente.

• No se importan main.main, auth_service.main ni el módulo de conexiones; se bloquearon las funciones de conexión en esa comprobación.

Qué falta demostrar

El test de modelos no abre una Session ni inserta, recupera o elimina registros. No hay una prueba de persistencia que compare los modelos con una base de pruebas migrada. Esa comprobación debe añadirse antes de trasladar el CRUD, es decir, las operaciones de crear, consultar, actualizar y eliminar.

Las comprobaciones de carga ASGI y rutas se ejecutaron en el mismo proceso con TestClient; no se certificó un servidor real escuchando en 8001. La inspección de PostgreSQL de esta revisión fue solo de estructura, separada de las pruebas automatizadas.

Las seis comprobaciones adicionales se ejecutaron durante la revisión y no se añadieron como archivos de pruebas permanentes. No se modificó código del proyecto.

08 / REVISIÓN ESPECÍFICA

Cómo ejecutar la preparación y qué sigue

Comandos desde la raíz del repositorio

```
.\.venv\Scripts\python.exe -m pip install -r publicaciones_service/requirements.txt
.\.venv\Scripts\python.exe -m uvicorn publicaciones_service.main:app --reload --port 8001
```

Abre **http://127.0.0.1:8001/health** o **http://127.0.0.1:8001/docs**. Para esta base independiente no se requiere DATABASE_URL. Su endpoint de estado comprueba el proceso, no el estado de PostgreSQL.

```
{"servicio": "publicaciones", "estado": "ok"}
```

Repetir las pruebas existentes

Para la suite completa se necesitan las dependencias de la aplicación principal y del ORM; no basta con asumir que requirements.txt del servicio incluye todas las herramientas de pruebas del repositorio.

```
.\.venv\Scripts\python.exe -m pip install -r main/requirements.txt
.\.venv\Scripts\python.exe -B -m unittest discover -s tests -v
```

Orden recomendado para la próxima etapa

Paso	Resultado esperado
1. Acordar esquema y migración	Precio, nulos, FK de imágenes y valores por defecto alineados.
2. Agregar conexión y sesiones propias	Configuración segura y transacciones sin depender de importar main.main.
3. Implementar persistencia	Operaciones con validación, listas actualizadas y política de imágenes definida.
4. Probar una base de pruebas	Insertar, leer, editar y borrar/cerrar sin afectar datos reales.
5. Trasladar rutas e integrar	Cambiar el consumidor solo después de conservar el contrato HTTP y la autorización.

Conclusión

Sí: la preparación del microservicio y sus modelos está bien hecha para la etapa descrita. No equivale a un servicio con persistencia terminada. El punto pendiente más importante es alinear los modelos con la base antes de usarlos para guardar datos; el resto de decisiones ORM está identificado en este informe.

Entrega de esta revisión: únicamente este PDF. Se conserva el informe anterior de editar, cerrar y consultar cupos; no se reemplazó ni se realizaron migraciones o escrituras sobre los avisos.